

AI Animation Pipeline — Technical Case Study

Julian Visser — Project Manager & Writer

visserjulian@gmail.com · [Portfolio](#) · [Source code](#) · 2026

About this document

This is a self-directed technical case study project. What follows walks through what I built, where I misjudged the scope, how I course-corrected and where I made mistakes. The point of publishing this isn't to indicate that the project is complete. It's to demonstrate the reasoning and the process.

A note on what "I built" means: I don't have formal training in computer science or software engineering, and I do not write production code from scratch. AI wrote the majority of the code in this repository. My job was to specify what to build, review what came back, catch errors before they compounded, and learn enough Python, JSON schemas, hexagonal architecture, atomic file operations, and CI to know when to push back on the tooling's suggestions. That is a different skill than writing production code from scratch, and this case study is about that skill.

1. The problem I picked

I set out to build a general-purpose animation pipeline. Prose or structured input in, animated content out, deterministic enough that I could control any stage, modular enough to work with any AI provider (image generation, image-to-video, vision judgment, TTS). The underlying premise was that AI-driven video generation is developing fast, current models aren't at professional standards yet. My hypothesis is that they will eventually get up to professional standards, same as TV/Films productions. My assumption is that the people, who prepare and have working AI pipelines, AI tooling and expertise ready when that professional grade AI-quality lands, will have an edge over studios that started learning the tools later. Especially if they know how to best utilize the AI tools.

Three assumptions I made at the time:

1. A distinctive visual aesthetic would function as a defensible market moat.
2. A broad platform was more useful than a narrow product.
3. A solo developer could carry the scope.

In hindsight, one and three were wrong. Two is arguable and depends on the audience.

2. Research context

Before writing code I went through research articles from the last two years. SIGGRAPH and NeurIPS talks, papers on ControlNet, AnimateDiff, and the Diffusion Transformer architecture underneath Sora, and industry positions from people working on the actual problem. What I read shaped what I built. What I read and *didn't* fully act on is more interesting for a case study.

People whose work informed the pipeline:

- **Lvmin Zhang** ([ControlNet](#)). Structural steering via edge, depth, and pose conditioning. Made ControlNet-style structure signals a first-class contract field in `piper/models/contracts.py` rather than an ad-hoc prompt argument.
- **Yuwei Guo** ([AnimateDiff](#), MMLab / CUHK). Motion modules and [Long Context Tuning](#) as the current path to scene-level temporal consistency. Informed the decision to treat image-to-video as a distinct pipeline stage with its own port, not a single-model call.
- **Sanja Fidler** (NVIDIA, Spatial Intelligence Lab). [World Foundation Models](#) as backbone for 3D-aware video. I noted this as a future direction; the current pipeline that I have made was for 2D design only, which the research would identify as the weaker of the two available architectural blocs. I assumed that 2D would be easier to work with as opposed to 3D images and videos. I was wrong.
- **Aaron Hertzmann** (Adobe Research). His position that AI tools don't become "artists" but do change how art is made and understood. This shaped how I write about the pipeline.
- **William Peebles and Saining Xie** ([Diffusion Transformer paper](#), the architecture underneath Sora). The claim that scaling this architecture surfaces "intuitive physics" emergently — made in [OpenAI's Sora report](#), not the DiT paper itself — set my expectations for what current models can and can't be asked to do without explicit physics constraints.

Findings that mapped directly to design decisions:

- **Small curated datasets outperform large scraped ones.** A 2025 arXiv paper (Akarsu et al., [2510.27364](#)) fine-tuned Wan2.1 I2V with LoRA on a single GPU using about 40 short clips (roughly 25,000 frame-caption pairs, or ~16 minutes of hand-cleaned footage) from the Turkish series *El Turco* to achieve cinematic domain transfer. I respected this finding in principle and did not act on it in practice. No curated LoRA was ever trained. It is the most consequential thing I read and did not follow through on.
- **Modular ("Diffusion Bloc") vs. black-box ("E2E Bloc") as the strategic choice.** I built the modular bloc. The same research also warned that a modular pipeline without proprietary data has no defensible moat. That warning was accurate.

- **Deterministic seeds and reproducibility as production-readiness gates.** Shaped the contract's insistence on explicit seeds and version-locked LoRAs.
- **USD and multi-layer EXR interchange as the "integration floor" for studio compatibility.** In hindsight, I did not build for this. This was a real gap if the target audience is studios rather than indie creators.

Where the research contradicted what I actually built. The research consistently said *start narrow*. Pick one style, curate one dataset, train one LoRA, ship. I built horizontal instead: a modular framework aimed at many styles and providers. This is the specific mismatch that my later recommendations for a pivot has been correcting. The Wētā position that proprietary data is the moat is also the opposite of what a solo developer using public models is doing. Not automatically wrong for a portfolio project, but the framing "modular pipeline as competitive edge" was never going to hold.

What I regret about the research process itself. I read through the research articles but I did not try and contact any of the people named above. On paper this looks like research; in practice it's only half of it. One 30-minute conversation with any working practitioner would have flagged the "build narrow first" recommendation early enough to save months of potential work. The reason I didn't reach out is that building felt like progress and outreach felt like risk. That instinct was wrong. On future projects I plan to reverse the order: outreach first, build second.

3. What I built

The pipeline is organized around a hexagonal ports and drivers pattern. External capability (image generation, video generation, vision-based consistency checking, asset vaulting) is defined as Protocol interfaces in `piper/ports/`. Concrete integrations live in `piper/drivers/`. A registry reads config and environment variables at startup and instantiates the right driver at runtime. Adding a new AI provider is one new driver module plus one registry entry.

Around this sit four things worth calling out.

Contract-driven pipeline. Requests are Pydantic models: `ImageContract`, `VideoContract`, `ShotBrief`, with strict validation. Business-rule validation runs before any paid API call. Ban-phrase enforcement catches known style violations (gradients, shadows, 3D shading) before they ever hit a provider. Cheap to enforce, expensive to skip.

Atomic writes and file locking. The production log is the single source of truth for what exists in the project. Writes go to a `.tmp` file first, then atomic rename. Advisory locks with stale-lock recovery handle concurrent access. This mattered more than I expected: the project lives on OneDrive, which occasionally interferes with atomic operations, and I hit real corruption early on before hardening the write path.

Multi-integration surface. Four ways to invoke the pipeline: CLI (`piper-run`), MCP server (for use inside Claude Desktop), FastAPI control plane (for orchestration), and a Streamlit UI (for operators). In hindsight this was over-scope for a project without users. I don't recommend the platform-first approach for a solo project. Each surface added maintenance debt without adding conversion probability. I kept them because ripping them out would be busywork, but I stopped adding to them.

Testing. Integration-style tests — 1,711 passing in the full private tree, 1,695/1,691 in the public snapshot (the delta is asset-gated skips, explained in the appendix). CI gates on formatting (ruff), type checking (ty), and pytest. Every commit runs the full suite before being accepted. Test discipline was worth the cost. It's the only reason I could re-scope the project later without breaking what had already worked.

4. What went sideways

Two categories. Both together mattered more than either alone.

Technical dead-ends

The pipeline kept surfacing problems that weren't downstream of a paying customer. A few examples:

- **The base model drifted to 3D no matter what the prompt said.** The style target is flat 2D vector — flat color regions, no gradients, no shading. Early live smoke tests came back glossy and isometric anyway: specular highlights, gradient shading, depth-of-field blur. Prompt-side ban phrases weren't enough; the fix that held was a style LoRA plus ban-phrase validation *before* the API call. The pair below shows the drift and the correction. These are two separate smoke-test runs against the same style target — not a re-render of the same shot — so compare the rendering style, not the composition.

Before: first live smoke	After: live smoke with style LoRA
	

- **Flux Canny needed contiguous edges.** Multi-part rigs (where a character's head, body, and limbs were separate assets) caused regressions of over 100% on drift metrics because Canny couldn't form a coherent contour. The fix was pre-merging parts via dilation and contour-fill before running edge detection. A day of diagnosis, a small code change, and a real lesson about how the underlying model interpreted its inputs.
- **Photo-trained segmentation models fail on flat vector art.** There's no texture, no gradient, no color variation for them to lock onto. A model trained on photographs of the world doesn't generalize to flat single-color regions. Diagnosing this took longer than it should have because I assumed general-purpose models would generalize. They don't.
- **Multi-subject scenes dropped subjects at generation time.** Single-subject compositions worked at production quality. Multi-subject compositions consistently lost fidelity downstream. Isolating this took paid experiments, and the fix isn't in the current release. It's a downstream architecture problem, not a prompt problem.
- **A rigging bug that taught me something.** At one point I had a pipeline storing ~200px silhouettes padded inside 1024×768 transparent canvases. Standard 2D affine transformations rotate around the origin. Forcing the bounding box to span the entire frame locked the pivot point to the canvas center (512, 384) instead of the part's natural anatomical center. Parts would "orbit" instead of rotating locally. The fix was localized bounding-box cropping, which turned into an architectural prerequisite for anything downstream. I include it here because it's the kind of failure that only shows up when you try to compose the pieces together, and it wouldn't have shown up in a test suite that only exercised the pieces in isolation.

Strategic misjudgment

I was building a platform without a target buyer. Cold-outreach territory, no warm intros, no committed budget for the training data that was supposed to differentiate it. The moat assumption didn't hold either. If your differentiator can be reproduced in a weekend by someone else with the same tools, it isn't a differentiator.

Scope misjudgment

A solo developer can't credibly maintain four integration surfaces for a product with zero users. This was compounded by the fact that adding a new surface always felt productive. It looked like progress. It was actually load-bearing debt.

Process misjudgment

Partway through, I discovered I had two parallel builds of the same pipeline running at the same time. I'd been iterating with automated coding assistance in one branch while making manual edits in another, and I wasn't paying enough attention to which was authoritative. Reconciling them cost a chunk of time I didn't have. Consensus between me, the tools, and any external contributors is not optional. It's a prerequisite. This one is worth naming because it's the kind of mistake that only happens once you have enough velocity to make it, and I want the record of it public rather than quietly cleaned up.

5. The salvage

I re-scoped from "general pipeline" to a narrow product: prose script in, short animated storyboard out. This makes it more concrete. Demoable. What the salvage proved — and it's the useful finding for a portfolio — is that a modular architecture lets you cut scope without starting over.

What I preserved. The Flux + Canny + LoRA + judge primitives from the original work were reorganized into a shared `piper/synthesis/` module and consumed by the new product. The rigging code wasn't thrown away. It stopped being the top-level product and became infrastructure for a smaller one. Clean architecture pays dividends in the wrong direction too: even a mis-scoped project can be salvaged if the internals were built cleanly.

What I added. A five-stage pipeline (parse, block, render, audio, video). New ports for text-blocking, narration, and music. A hard cost ceiling (`PIPER_ANIMATIC_ALLOW_COST` , default \$5.00) so a runaway loop can't drain my credit card. A doctor pre-flight command that catches environment problems (missing API keys, disk space, unresolvable paths) before a run starts. All of these came from watching real failure modes in the earlier work.

Where the salvage stopped. Architecture and test coverage are in place. Live-API validation on real Fal, ElevenLabs, and Anthropic endpoints was the next milestone, and I stopped before that. Continuing this specific product further would need real customer discovery paired with an animation professional who has the studio contacts I don't currently have. That is a partnered venture, not more solo engineering. This case study is the deliverable.

6. What I learned

Scope discipline. A solo developer's runway is measured in weeks of attention, not years. Multi-surface platforms are for teams with users. Next time I'll validate one narrow demo before building infrastructure around it. This is the single most transferable lesson from the project.

Sunk-cost hygiene. Clean architecture is what let me salvage the original build. Ports and Protocols made the reorganization into a narrower product much cheaper than starting over. If the internals had been tightly coupled, the re-scope would have meant a rewrite. It actually meant a reorganization instead.

Calibrated AI over agreeable AI. The most productive change I made to my own tooling was insisting the AI tell me when it was uncertain, when it thought my premise was wrong, what alternatives it saw, and what could go wrong with my chosen path. Not agreement. Not encouragement. Confidence labels, per-claim assumptions, skeptical review of my own reasoning. This reshaped what I got out of the AI tooling more than any prompt engineering did.

Systemize verification between stages. AI is very good at chasing whatever goal you set and less good at checking whether the goal makes sense downstream. If you don't put explicit verification between pipeline stages, you get plausible-looking output that breaks under close reading. Multi-stage validation, ban-phrase enforcement, cost ceilings, and the doctor pre-flight all came from watching that pattern play out in real runs.

Costed experiments. Cumulative spend on paid smoke tests during the rigging work: around \$3.25. Small in absolute terms. The discipline of setting explicit cost ceilings and warning thresholds is one I'll carry into any future ML-tooling project. Watching small runs add up teaches respect for compound costs faster than any spreadsheet does.

Learning without a curriculum. Naming what I didn't know (async, Pydantic v2, ffmpeg determinism, atomic file operations on Windows, hexagonal architecture, CI hygiene) and then learning it in situ produced deeper retention than a structured course would have. It also took longer per skill acquired. Both facts are true. Which one matters more depends on what you're optimizing for.

Appendix: what to look at in the repo

The public snapshot lives at <https://github.com/Vhaser-system/piper-animatic> — a frozen "portfolio v1" export that installs standalone and passes its own test suite (1,695 passed / 25 skipped on Windows; 1,691 / 29 on the repo's own Linux CI, where ffmpeg-dependent tests run in a dedicated green job). The delta from the 1,711 tests passing in the full private tree is asset-gated tests that skip themselves where the privately retained training data and run artifacts are absent — a skip, never a failure. Suggested reading order:

- `piper/ports/` — the Protocol interfaces that make every external capability swappable
- `piper/drivers/` — the concrete implementations behind those ports (Fal, Replicate, Anthropic, OpenAI, vtracer, ffmpeg)
- `piper/drivers/registry.py` — where config decides which driver runs; adding a provider is one module plus one registry entry
- `piper/models/contracts.py` — the contract schema that drives batch runs
- `piper/validation/validator.py` — business-rule enforcement, including the style ban-phrases
- `piper/vault/` — the production log: advisory locking, WAL-style atomic writes
- `piper/animatic/` — the salvaged product: prose → storyboard animatic
- `piper/synthesis/` — the closed-loop generate-judge-retry runner
- `projects/flat_2d/` — the project pack: policy, style profile, rigging templates
- `tests/` — 1,700+ integration-style tests; the suite is the honest map of what actually works

Every claim in Section 3 traces back to a specific module. Every failure in Section 4 traces back to a specific fix. If any of it looks fabricated, the full development history is retained — ask, and we'll walk through it together.